

NAME - DAKSH PHANWANI

ENROLL NO. → 0801EI231031

SUBJECT → EMBEDDED SYSTEM DESIGN

ASSIGNMENT → 01 + 02

ASSIGNMENT-01

Q1. Define →

a) Watchdog Timer → A watchdog timer is a hardware timer used to detect and recover from system malfunctions.

- It resets the system if the software fails to periodically refresh (kick) the timer.

- In embedded systems, it ensures reliability and fault tolerance, especially in critical applications like automotive or medical devices.

b) Semaphore → A semaphore is a synchronization mechanism used to control access to shared resources in multitasking systems.

- It prevents race conditions between tasks.

- Types → Binary and counting semaphore.

- In embedded systems, it is widely used in RTOS for task coordination.

c) Deadlocks → A deadlock is a condition where two or more tasks are unable to proceed because each is waiting for a resource held by another.

- It results in system halt or freeze.

- In embedded systems, deadlocks must be avoided as they can cause critical system failure.

d) Multitasking → It refers to the ability of a system to execute multiple tasks concurrently.

- Achieved using scheduling algorithms in RTOS.

- In embedded systems, it enables efficient use of CPU for handling tasks like sensing, processing, and communication simultaneously.

e) Priority Inversion → Priority inversion occurs when a higher-priority task is blocked by a lower-priority task holding a ~~shared~~ shared resource.

- It can degrade real time performance.

- Solved using techniques like priority inheritance in embedded systems.

f) Context Switching → It is the process of saving the state of one task and loading the state of another task.

- Enables multitasking in embedded systems.

- Includes saving registers, program counter and stack.

- It introduces overhead, so it must be optimized in real-time system.

Q2. A real-time - - - - - steps.

Ans.)

→ It is a Hard Real-Time System.

- Because timing deadlines (like signal switching) must be strictly met.

- Delay can lead to traffic congestion or accidents.

→ Required Characteristics:-

The system should have →

- Deterministic behaviour → Predictable response time.

- Low Latency → Immediate reaction to traffic conditions.
- High Reliability → Continuous operation without failure.
- Fault Tolerance → Backup mechanisms (e.g. watchdog timer).
- Concurrency → Handle multiple signals and sensors simultaneously.
- Scalability → Adapt to increasing traffic and smart city features.

→ Design Process Steps:-

1. Requirement Analysis
 - Identify inputs (traffic sensors, cameras)
 - Define timing constraints and safety requirements
2. System Specification
 - Define system architecture (microcontroller, sensors, communication modules).

3. Hardware-Software Partitioning :-

- Decide which tasks are hardware-based (timers, interrupts) and software-based.

4. RTOS Selection

- Choose RTOS for multitasking and scheduling.

5. Design & Implementation

- Develop control algorithms for signal timing.
- Implement scheduling (priority-based).

6. Testing and Validation

- Verify timing constraints and system response.
- Simulate real-traffic scenarios.

7. Deployment and Maintenance

- Install in real environment.
- Monitor and update system periodically.

ASSIGNMENT-02

Q/r Diff. b/w microcontroller and microprocessor.

Ans.

- **Microprocessor:** This is essentially just the CPU on a single chip. It does not have built-in RAM, ROM or input/output (I/O) ports. These peripherals must be added externally on the motherboard. They are designed for general-purpose, complex ~~comp~~ computing tasks and operate at high clock speeds, consuming more power.
- **Microcontroller:** This is a complete computer on a single chip. It integrates the CPU, RAM, ROM and I/O ports all within the same Integrated Circuit (IC). They are designed to perform specific, dedicated tasks (like in washing machines or embedded automotive systems), operating at lower clock speeds (MHz) and consuming minimal power.

Q/2. What is Arduino? List its key features?

Ans:

Arduino is a open-sourced electronics platform based on flexible, easy to use hardware and software. It consists of a physical programmable circuit board (often referred to as microcontroller) and a piece of software or IDE, that runs on your computer to write and upload computer code to the physical board.

→ Key Features :-

- Open-source hardware and software.
- Easy programming using Arduino IDE.
- Built-in I/O pins for interfacing.
- Supports sensors and modules.
- Low cost and beginner-friendly.
- Cross-platform support.
- Large community support.

Q3. Explain the Architecture and Functional block diagram of Arduino.

Ans: The architecture of a standard Arduino is based on the Atmel AVR microcontroller (ATmega328P). It follows the Harvard architecture, where program memory and data memory are separate.

- CPU: The brain that fetches and executes the instructions.
- Memory: Non-volatile memory where the Arduino sketch is stored.
 - SRAM: Volatile memory where the sketch creates and manipulates variables while running.
 - EEPROM: Non-volatile memory used to store long term data that persists even when the board is powered off.
- Digital I/O: pins used for reading digital sensors or driving digital outputs.
- Analog Inputs: connected to built-in ADC to read variable voltages from sensors.
- Communication ^{ports} ~~peripherals~~: - built-in hardware ~~to~~ for

serial communication (UART, I2C, SPI) to talk to computers or other modules.

Q4 Compare ----- applications.

Feature	Arduino	Raspberry Pi
Architecture	Microcontroller based (no O.S., runs one compiled loop).	Single board computer (microprocessor based, runs a full linux O.S.).
Performance	Low performance. Typically 16MHz clock speed, a few kilobytes of RAM.	High performance. Typically >1GHz CPU, 1GB to 8GB of RAM.
Applications	Real-time control, precise timing tasks, simple repetitive logic, reading basic sensors, low-power battery operations	Heavy processing tasks, computer vision, web servers, running multiple complex scripts concurrently, IOT gateways.

Q/s.

Explain PWM - - - - - control.

Ans.

In Arduino, PWM is easily generated using the `analogWrite(pin, value)` function. The pin must be one of the designated PWM pins. The value is an integer between 0 and 255, dictating the duty cycle.

- `analogWrite(pin, 0)` generates a 0% duty cycle (always off, 0V).
- `analogWrite(pin, 127)` generates a 50% duty cycle (half on, half off, avg 2.5V).
- `analogWrite(pin, 255)` generates a 100% duty cycle (always on, 5V).

→ DC motors spin at a speed proportional to the voltage applied to them. Since an Arduino's digital pins can only output 0V or 5V, it cannot gradually change a motor's speed directly. By using PWM, the Arduino sends a rapidly pulsing signal to a motor driver circuit. If the duty cycle is set to 50%, the motor driver delivers half the maximum average voltage to the motor, causing it to spin at half its maximum speed. This allows for smooth acceleration and precise speed regulation in robotic applications.